

CODE SPORT



In this month's column, we continue our discussion on the threading support introduced in the new C++11 standards. We first look at how we can manage threads.

In last month's column, we had discussed how to use `std::thread` to create threads and how `std::mutex` can be used to protect shared data from concurrent accesses from multiple threads. While it is possible to use mutexes to protect global variables, the general practice is to group the mutex and the data it protects together into the same class. The member functions of the class use the appropriate mutex and access/update the shared data. The intent is to hide the implementation of the synchronised access to the protected data inside the class. However, the burden of making sure that data is appropriately protected by locks and is always accessed after acquiring the required locks is on the programmer and is not automatically enforced by the language. For instance, consider the following example:

```
class my_data {
private:
    //members..
public:
    //do_somework does some sensitive operation,
    //This requires that proper locks need to be
    //held since concurrent threads can execute
    //this
    void do_somework();
}
Class my_data_wrapper {
Private:
```

```
my_data data1;
std::mutex my_data_mutex;
template<typename Function> void
PerformOperation(Function my_func)
{
    my_data_mutex.lock();
    my_func(data1);
    my_data_mutex.unlock();
}

//now consider an unprotected use of my_data
my_data* unprotected;
void bad_func(my_data& data1)
{
    unprotected = &data1;
}
my_data_wrapper d1;
void foo()
{
    my_data_wrapper A;
    //here we expose the protected data using a
    //pointer reference
    A.PerformOperation(bad_func);
    //now we can circumvent the my_data_wrapper
    //and invoke the functions on the unprotected
    //data

    unprotected->do_somework();
}
```

What is wrong with the above piece of